MinAI — The Trainer's Guide

A single-file C++ transformer that learns to reverse `[0..7] → [7..0]`. 1,216 to 4,032 parameters depending on config. Ancestry: Damien Boureille's ATTN-11 (PDP-11 MACRO-11 assembly) → Dave Plummer's 2.11BSD port → this repo (Apple-Silicon C++).

> **New to this and want the long explanation?** Read **GUIDED_TOUR.pdf** (generated from `GUIDED_TOUR.md`) first. It is a textbook-style walkthrough of every concept the code uses — what a model is, what attention does, what calculus is actually for, and how the whole thing trains — written for a reader who has taken some math in school but has never seen *why* any of it mattered. A bright high-schooler can follow it. The last chapter is a **hands-on walkthrough of the flags**: run a command, see what it teaches, run the next one. This file (TRAINER.md) is the terse operator's manual; `GUIDED_TOUR.md` is the "what is going on under the hood" companion. Read them side by side with `minai.cpp` open and you will actually see the curtain pulled back.

Quick start

```
make              # builds ./minai with clang++ -O2
./minai           # run with defaults
./minai --help    # flag reference
make clean        # remove the binary
```

The run prints training progress every 50 steps, then a demo showing the trained model reversing `0..7`, then the last block's attention matrix, then a bonus Q8 fixed-point softmax demo (the actual PDP-11 trick, no `std::exp`).

Flag reference

Flag	Values	Default	What it does
`--blocks=N`	1..96	`1`	Stack N transformer layers. Each layer is attention + (optional) FFN. `--blocks=96`

Flag	Values	Default	What it does
			matches GPT-3's layer count exactly. Stacks deeper than ~16 often fail to train without LayerNorm — a useful failure to witness.
`--ffn=0|1`	`0` or `1`	`1`	Include the feed-forward sub-layer inside each block. Off = pure attention.
`--causal=0|1`	`0` or `1`	`0`	Mask attention so position `t` can only see positions `≤ t`. This is what makes a transformer "generative".
`--layernorm=0|1`	`0` or `1`	`0`	Pre-LN normalization before each sub-layer (GPT-2+ style). The thing that makes deep stacks trainable — without it, `--blocks=32` produces `NaN`; with it, even `--blocks=96` trains.
`--random=0|1`	`0` or `1`	`0`	`0` = train on the single fixed example `[0..7]`. `1` = fresh random sequence every step, with a frozen held-out eval set so you can watch generalization.
`--seq_len=N`	1..32	`8`	Sequence length. Attention cost is `O(N²)` in this, so doubling it ~4×'s the attention time per block.
`--batch=B`	1..128	`1`	Examples per training step, averaged into one gradient update. Bigger batch = smoother loss curve, but each step is proportionally slower.

Flag	Values	Default	What it does
`--task=NAME`	`reverse` / `sort` / `shift` / `mod_sum`	`reverse`	Which target rule the model has to learn (see **Task zoo** below).
`--steps=N`	≥1	`800`	Total training steps.
`--extra_demos=0|1`	`0` or `1`	`0`	After training, run three bonus inference-side demos: hierarchical weight quantization, speculative decoding, and KV cache tiering. Each is a real production LLM technique, shown in miniature. Adds a few seconds of runtime.
`--help` / `-h`	—	—	Print usage and exit.

The three bonus demos (`--extra_demos=1`)

These run after training is done and give you a small-scale taste of three techniques real LLM inference stacks rely on:

- **Hierarchical weight quantization** — takes the trained `Wout` matrix and stores it three ways: Q8 flat, Q4 flat, and Q4 with a per-column "feet + left hand + right hand" scale hierarchy. Prints max / RMS reconstruction error and whether the model still works with the quantized weights. The hierarchical Q4 at ~5 bits/weight ends up between Q8 and flat Q4 in precision — which is the trick behind llama.cpp's Q4_K_M format that lets a 70B model run on a MacBook.
- **Speculative decoding** — trains a second smaller "draft" model (half the block count of the big one, with everything else on) and measures how often it agrees with the big model on the held-out set. The agreement rate is the expected acceptance rate in real speculative decoding, and translates directly to a throughput speedup (the big model verifies K draft tokens in one forward pass — most of its time is spent loading its own weights, so amortizing that over multiple accepted tokens is ~pure win).
- **KV cache tiering** — sweeps a "hot working memory" size W from 0 to `seq_len` and re-runs the forward pass with K/V at every position older than W quantized to Q8. Reports the accuracy at each W, with Miller's 7 (the classic human working-memory capacity) called out. On MinAI specifically the effect is subtle because Q8 is so fine-grained; the note in the output explains what real LLMs do differently.

The compile-time upper bounds `MAX_BLOCKS`, `MAX_SEQ_LEN`, `MAX_BATCH` live in Part 1 of `minai.cpp`. Bump and recompile if you want bigger runs.

Task zoo

Four target rules, all operating on an `N`-digit input:

`--task=`	Rule	Needs attention?	What it teaches
`reverse`	`target[t] = input[N-1-t]`	Yes — each output retrieves input `N-1-t`	Pure long-range attention.
`sort`	`target = sorted(input) ascending`	Yes, and more of it — each output needs global info to decide its rank	Harder than reverse; often needs more blocks.
`shift`	`target[t] = input[(t+1) mod N]`	Barely — just positional	Easy; attention doesn't do much work.
`mod_sum`	`target[t] = (input[t] + input[(t+1) mod N]) mod 10`	Local (two adjacent positions)	FFN does most of the work; nearly attention-free.

Example runs

```
./minai                                          # defaults: 1 block, FFN on, fixed [0..7]
./minai --blocks=1 --ffn=0                      # classic 1,216-param MinAI
./minai --blocks=2                              # stack two full layers
./minai --random=1                              # random sequences + held-out accuracy
./minai --random=1 --causal=1                   # watch causal masking actually hurt
./minai --random=1 --blocks=2 --batch=16        # real-world-ish training setup
./minai --seq_len=16                            # longer inputs (attention is O(N^2))
./minai --task=sort --random=1 --blocks=2       # harder task, needs more capacity
./minai --task=mod_sum                          # local task; FFN does the work
./minai --blocks=32 --layernorm=0 --steps=200   # deep + no LN → NaN (gradients explode)
./minai --blocks=32 --layernorm=1               # same depth + LN → trains smoothly
./minai --blocks=8 --layernorm=1 --random=1 --batch=16  # 100% held-out on random reversal
./minai --blocks=96 --layernorm=1 --steps=200   # GPT-3 depth, actually trains (slow)
```

The loss curve plot

At the end of every run MinAI prints an ASCII sparkline of the per-step loss, plus (if `--random=1`) a sparkline of held-out accuracy over training. Example:

```
Loss curve (800 steps, 60 columns, higher bars = higher loss):
  2.280 |▇▇▇
```

The first 20 training steps are logged verbatim (every step) so you can see the raw early numbers; later steps report less frequently.

Terminology — what "1 block, FFN on" actually means

Different papers use different words for the same thing. Here is the one-to-one dictionary:

- **"block" = "layer" = "transformer layer" = "transformer block"**. They all mean the same unit: one pass of attention followed by some per-token transformation, wrapped in residual connections. GPT-3 has 96 of these. MinAI with `--blocks=N` has N of them. `--blocks=1` is one transformer layer.
- **"FFN on" = the feed-forward sub-layer is included inside each block.** A *canonical* transformer block is two sub-layers stacked:

  ```
  block:
   H1 = X  + attention(X)     ← sub-layer 1: attention + residual
   H2 = H1 + FFN(H1)          ← sub-layer 2: FFN + residual  (turned off by --ffn=0)
  ```

  Attention moves information **across** positions. FFN transforms information **within** each position (it's a 2-layer MLP applied to each token independently). A canonical block has both.
- **`--ffn=0` strips off the second sub-layer**, leaving each block as just attention + residual:

  ```
  block:
   H = X + attention(X)       ← the whole block
  ```

  Still trains, still converges — but it's "half a layer" in the canonical sense.

So the config names decode as:

Flag combo	Plain English
`--blocks=1 --ffn=1` *(default)*	One full transformer layer.
`--blocks=1 --ffn=0` *(classic MinAI)*	Half of one layer — attention only.
`--blocks=2 --ffn=1`	Two full transformer layers stacked.
`--blocks=2 --ffn=0`	Two attention-only half-layers stacked.
`--blocks=N --ffn=1 --causal=1`	N stacked layers of a GPT-style decoder.

When a paper says "GPT-3 is 96 layers", they mean 96 of the `--ffn=1` kind.

What each configuration teaches

These are approximate numbers from deterministic runs (the RNG seed is fixed so yours will match). Every config achieves 8/8 correct — the lessons are in the *loss* and in the *attention matrix*.

Command	Params	Final loss (step 800)	What to notice
`./minai --ffn=0`	1,216	~0.022	The classic MinAI. Attention is fuzzy; the output projection does most of the work.
`./minai`	2,240	~0.009	Adding FFN cuts loss 2–3×. Same attention pattern, but the FFN sharpens the per-token output.
`./minai --blocks=2`	4,032	~0.002	Loss drops another 4×. Block 1's attention starts showing *actual reversal*: row 6 and row 7 put ~0.6 weight on input 0, as they should.
`./minai --causal=1`	2,240	~0.005	Upper triangle of attention is exactly 0. Still 8/8 — but see "gotcha" below.

The causal gotcha. With causal masking on and our *fixed* input sequence, training still converges to 8/8. Not because the model learned to reverse — it can't, position 0 cannot see the last input — but because each output position can simply *memorize a constant*. This is a classic machine-learning anti-lesson: **a trivial dataset can make a broken architecture look fine.** To see causal masking actually bite, you'd need varied inputs so per-position memorization stops working.

Why this program exists

When you read a paper about a 175-billion-parameter model with 96 layers of multi-head attention, it is easy to lose the plot. **This program is the plot.** Everything a large transformer does, this program does too. It just does it *once* instead of 96 times, with a few thousand parameters instead of 175,000,000,000.

If you fully understand these ~540 lines, you understand the skeleton of every frontier model.

The conceptual map: why this one program is “the whole plot”

Here’s the trick I want to drill into your head, because once you see it, the scale-up stops being mysterious. **Every transformer on earth, including GPT-4, is this program applied repeatedly.** Literally. The list of things GPT-4 adds:

What MinAI has (flaggable)	What GPT-4 adds on top
1–4 attention blocks (`--blocks=N`)	~96 of them stacked
1 attention head per block	~96 heads per block, run in parallel
optional FFN per block (`--ffn=1`)	always-on FFN, `D_FF = 4 × D_MODEL`
optional causal mask (`--causal=1`)	always-on causal mask (for decoder models)
`D_MODEL = 16`	`D_MODEL ≈ 12,288`
`VOCAB = 10`	`VOCAB ≈ 100,000` (BPE subwords)
no LayerNorm	LayerNorm before each sub-block

No new ideas in that list. More layers, more heads, more width, a normalization trick. The skeleton is identical to what you just compiled and ran.

The five concepts, in the order they appear in the forward pass

1. Embeddings (Part 4 + step 1 of forward)

Neural nets can’t do arithmetic on the symbol “3”. So we keep a lookup table: `token_emb[3]` is a learned 16-dimensional vector. And because pure attention is order-blind (swap tokens 1 and 5 → identical output), we *add* a learned position vector `pos_emb[t]` for each slot. That single addition is how the model tells “digit 3 at position 0” apart from “digit 3 at position 5”.

2. Attention (Part 4 + steps 2–5 of `forward_block`)

The mechanism is a soft dictionary lookup:

- Each position emits a **query** Q (what I’m looking for), a **key** K (what I advertise), and a **value** V (what I contribute if chosen).
- “How well do I match you?” = dot product of my Q with your K.
- Softmax those scores across all positions → a distribution: `attn[t][·]` is “how much I’m paying attention to each position”.

- My new representation is `Σ_u attn[t][u] · V[u]` — a weighted average of V's.

For this task, we *hope* the model learns `attn[t] ≈ one_hot(7-t)`, so position t literally picks up position 7-t's V and ships it to the output. Look at the attention matrix the program prints — in a 1-block run it's fuzzy (because the residual path means the model can also reverse via the output projection); in a 2-block run the second block's attention begins to look like actual reversal. That is a useful lesson on its own: **there's rarely one "correct" internal solution**.

3. Softmax (Part 7)

Turns any real-valued vector into a probability distribution. Two things worth really getting:

- It's translation-invariant: `softmax(x) == softmax(x + c)`. That's why the "subtract the max first" trick works — it prevents `exp(100) = inf` without changing the result.
- In the forward pass we call `std::exp`. **On the PDP-11 there is no `exp`.** The final part of `minai.cpp` (Part 15) shows what they did instead: a 256-entry table of `exp(-k/8)` stored as Q8.8 integers, indexed by `(max - x[i]) >> 5`. You can watch it run at the end of program output — matches float softmax to about three decimals. That is how "AI" was done on hardware with no FPU.

4. Cross-entropy loss + the magical gradient (Part 9 + step 1 of backward)

The loss is `-log(probs[target])`, averaged over positions. The magic:

```
d(cross_entropy(softmax(logits))) / d(logits) = probs - one_hot(target)
```

Just that. No exp, no log in the backward direction. This identity is why every classifier in the world stacks softmax + cross-entropy together — the algebraic cancellation is too good to pass up. In the code, that's *one line* in `backward()`. Everything downstream is chain rule.

5. Backprop = chain rule, applied one op at a time (Part 10)

The whole backward pass is: for each forward op, given the gradient at its output, compute the gradient at its inputs and its weights. For a linear layer `Y = X @ W`:

```
 dX = dY @ Wᵀ
dW = Xᵀ @ dY
```

Memorize that pair. It appears many times in `backward_block()` (output projection, Q projection, K projection, V projection, `W1`, `W2`). The rest of backprop is:

- **Addition**: gradient passes through unchanged to both inputs. (Residual connections.)
- **Softmax**: `dx_i = y_i · (dy_i - Σ_j y_j · dy_j)`. (Attention softmax.)
- **Scalar multiply**: gradient gets multiplied by the same scalar. (The `1/sqrt(D)`.)

- **ReLU**: gradient passes through where input was positive, zero elsewhere. (FFN.)
- **Lookup**: gradient accumulates at the looked-up slot. (Embeddings.)
- **Mask**: gradient is zero in masked positions. (Causal mask.)

That is *the complete rule set*. Every neural net you will ever see is these patterns composed.

Stacked blocks (`--blocks=N`)

"Block" and "layer" are the same thing — one unit of attention + (optional) FFN, wrapped in residuals (see the Terminology section above). Stacking N blocks means: take the output of block `b`, feed it in as the input to block `b+1`. The first block sees the raw embedding; the last block's output goes to `Wout`. `--blocks=N` is equivalent to "N transformer layers".

Each block has its *own* `Wq, Wk, Wv, W1, W2` — they are not shared. So a 2-block model has twice the per-block parameters. The backward pass walks blocks in *reverse* order (block N-1 back to block 0), passing each block's `dX_in` as the next block's `dX_out`. That's the entirety of what "deep network training" means.

Why stack? Because one block can do one step of reasoning. The deeper you stack, the more *compositional* the computations the model can perform. A 2-block model solving reversal can, for instance, use block 0 to "align each position with its mirror" and block 1 to "actually fetch the mirrored value" — this is speculative interpretation, but something like it is visible in the learned attention matrices of real models.

The feed-forward block (FFN) (`--ffn=1`)

The FFN is the "per-token thinking" part of a transformer. Attention moves information *across* positions; the FFN transforms information *within* a position. It's a two-layer MLP applied to every token independently:

```
 ff_pre = H1 @ W1          # expand:   D_MODEL (16) → D_FF (32)
ff_act = relu(ff_pre)     # nonlinearity
ff_out = ff_act @ W2      # project:  D_FF (32) → D_MODEL (16)
H2     = H1 + ff_out      # residual
```

Why `D_FF > D_MODEL`? Because the nonlinearity in the middle is what gives the network expressive power. The wider the middle, the richer the transformations you can represent. In GPT-3 it's `D_FF = 4 × D_MODEL`. In MinAI we use `2×` to keep the parameter count honest.

Roughly two-thirds of the parameters in GPT-3 live in FFN layers. Attention gets the headlines but the FFN is where most of the math lives.

With `--ffn=0`, a block is just `H = X + attn_o` — pure attention with a residual. Still trains, still solves the task; the reversal simply doesn't need a per-token MLP.

The causal mask (`--causal=1`)

Before softmax in the attention step, we set `scores[t][u] = -∞` for `u > t`. The model literally cannot attend to future positions. Why do this? Because for *generative* language modeling you train by asking "given tokens 0..t-1, predict token t" at every position simultaneously. Without the mask, the model could cheat by peeking at the answer.

When you run `--causal=1` and look at the attention matrix the demo prints, the upper-right triangle will be exactly 0.00. That is the mask doing its job.

For our reversal task, causal masking is a disaster on paper — position 0 needs to output digit 7, which is the *last* input. But as noted in "What each configuration teaches" above, the fixed input lets the model cheat by memorizing constants per position. You'd see the mask actually break things if you generalized the task to varied inputs.

The Q8.8 / Q15 side story (why it's educationally useful)

The PDP-11 has no FPU. Boureille's implementation uses:

- **Q8.8** (`int16_t` , low 8 bits fractional): forward pass activations. Value `1.0` is stored as `256` . Precision 1/256.
- **Q15** (`int16_t` , 15 bits fractional): gradients. Precision 1/32768. Needed because gradients are tiny — in Q8.8 a gradient of 0.001 rounds to zero and training stalls.
- **Q16 in `int32_t`** : weight accumulators, so many small updates don't round to zero.

The elegant bit: Q8 × Q15 in a 32-bit register gives Q23; one arithmetic shift right by 8 puts it back to Q15. One instruction, no rounding surprises. That's the entire "trick" to doing machine learning without a floating-point unit. In modern terms: this is *quantization*, exactly what gets used today to shrink LLMs from 16-bit floats down to int8 or int4 for edge inference. The PDP-11 was doing it in 1975 because it *had* to; Apple's Neural Engine does it in 2025 because it's *faster*. Same trick, different reason.

What's next (the journey continues)

Once you have blocks + FFN + causal wired up (you do), the remaining gap to a "real" GPT-style model is:

1. **Multi-head attention** — split the Q/K/V projections into `H` heads, do attention independently in each, concatenate. Same math as one head, just parallel. ~30 lines.
2. **Layer normalization** — `LayerNorm(x) = (x - mean) / std * gain + bias` , applied before each attention/FFN sub-block. Stabilizes training as depth grows. ~20 lines + backward.
3. **Larger vocabulary + varied inputs** — random sequences instead of the fixed one. Now causal masking actually matters and the model has to generalize.

4. **BPE tokenization** — so it can handle words, not just single digits. Out of scope here; this is what turns “digit reverser” into “language model”.

Items 1 and 2 are pure mechanical additions. Item 3 changes what the task *is*. Item 4 is a whole different project. But steps 1 and 2 are the full distance from MinAI to “tiny GPT”.